DAY 06 OF 10

Functions

Write reusable blocks of code

Today's agenda

1. Defining and calling functions
2. Parameters and return values
3. *args and **kwargs
4. Scope
5. lambda, map, filter and recursion

Python in 10 Days
Lakshmi Prasad Gondinigari

Why functions?

DAY 06

- A function is a named block of code you can run many times
- Avoid repeating yourself (DRY principle)
- Makes programs shorter, testable and easier to read
- You already used built-in functions: print, len, range, int
- Define your own with def

Defining a function

- def name(parameters): then an indented body
- return sends a value back to the caller
- No return means the function returns None
- A docstring describes what the function does
- Default values make a parameter optional

Python

```python
def greet(name, greeting="Hello"):
    """Return a greeting message."""
    return f"{greeting}, {name}!"

print(greet("Asha"))
print(greet("Ravi", "Welcome"))
print(greet(greeting="Hi", name="Sam"))
```

OUTPUT

```
Hello, Asha!
Welcome, Ravi!
Hi, Sam!
```

Returning multiple values

- return a, b actually returns a tuple
- Unpack the result into separate variables
- Keep each function focused on ONE job
- Name functions with verbs: calculate_total, is_prime

Python

```python
def min_max(nums):
    return min(nums), max(nums)

lo, hi = min_max([4, 9, 1, 7])
print(lo, hi)

def is_even(n):
    return n % 2 == 0

print(is_even(10), is_even(7))
```

OUTPUT

```
1 9
True False
```

*args and **kwargs

- *args collects extra positional arguments into a tuple
- **kwargs collects extra keyword arguments into a dict
- Useful when you do not know how many inputs will come
- Order in the definition: normal, *args, keyword, **kwargs

Python

```python
def add_all(*nums):
    return sum(nums)

print(add_all(1, 2, 3, 4))

def show(**info):
    for k, v in info.items():
        print(k, "=", v)

show(name="Asha", age=21)
```

OUTPUT

```
10
name = Asha
age = 21
```

Scope: local vs global

- Variables created inside a function are local to it
- Variables outside are global and readable inside
- Assigning inside a function creates a new local variable
- Avoid the global keyword; pass values and return results instead

Python

```python
x = 10            # global

def change():
    x = 5         # local, separate
    print("inside:", x)

change()
print("outside:", x)
```

OUTPUT

```
inside: 5
outside: 10
```

lambda, map, filter, sorted

DAY 06

- lambda: a tiny one-line anonymous function
- map applies a function to every item; filter keeps items that pass
- sorted(..., key=...) sorts by any rule
- Comprehensions are often clearer than map / filter

Python

```python
nums = [1, 2, 3, 4, 5, 6]
print(list(map(lambda n: n * 2, nums)))
print(list(filter(lambda n: n % 2 == 0, nums)))

people = [("Asha", 21), ("Ravi", 19), ("Sam", 25)]
print(sorted(people, key=lambda p: p[1]))
```

OUTPUT

```
[2, 4, 6, 8, 10, 12]
[2, 4, 6]
[('Ravi', 19), ('Asha', 21), ('Sam', 25)]
```

Recursion

DAY 06

- A function that calls itself
- Always needs a base case that stops the calls
- Elegant for factorial, Fibonacci, trees; loops are often faster
- Python limits recursion depth (about 1000 calls)

Python

```python
def factorial(n):
    if n <= 1:        # base case
        return 1
    return n * factorial(n - 1)

print(factorial(5))

def fib(n):
    return n if n < 2 else fib(n - 1) + fib(n - 2)

print([fib(i) for i in range(8)])
```

OUTPUT

```
120
[0, 1, 1, 2, 3, 5, 8, 13]
```

Practice time

DAY 06

1. Write is_palindrome(text) that ignores case
2. Write a function that returns the average of any number of values
3. Write count_vowels(text) and call it for 3 sentences
4. Write a function that returns a list of primes up to n
5. Use sorted with key to sort words by length

Challenge: write a function that validates a password (length, digit, uppercase)

Day 6 recap

DAY 06

- def name(params): ... return value
- Default, keyword, *args and **kwargs parameters
- Local vs global scope
- lambda / map / filter / sorted(key=)

Homework: finish practice. Next: Modules, Files and Exceptions